

源程序第 1 页 .gitignore

```
===== 文件开始 .gitignore =====
0001 # Python bytecode and caches
0002 __pycache__/
0003 *.py[cod]
0004 *$py.class
0005 .pytest_cache/
0006 .mypy_cache/
0007 .ruff_cache/
0008 .coverage
0009 htmlcov/
0010
0011 # Packaging and virtual environments
0012 *.egg-info/
0013 build/
0014 dist/
0015 .env/
0016 venv/
0017 env/
0018
0019 # Jupyter temporary files
0020 .ipynb_checkpoints/
0021
0022 # IDE and operating-system files
0023 .idea/
0024 .vscode/
0025 .DS_Store
0026 Thumbs.db
0027
0028 # Runtime logs and generated test outputs
0029 *.log
0030 test_output/
0031 output/*
0032 !output/.gitkeep
0033 !output/demo/
0034 !output/demo/.gitkeep
0035
0036 # Local configuration and environment secrets
0037 .env
0038 .env.*
0039 llmctg.local.json
0040
0041 # Local model weights and training checkpoints
0042 models/
0043 checkpoints/
0044 checkpoint-*/
0045 *.bin
0046 *.pt
0047 *.pth
0048 *.ckpt
0049 *.safetensors
```

源程序第 2 页 .gitignore

```
0050
0051 # Internal document rendering and QA intermediates
0052 qa/
0053 deliverables/*.pdf
0054 ~$*.docx
0055
0056 ===== 文件开始 pyproject.toml =====
0057 [build-system]
0058 requires = ["setuptools>=68"]
0059 build-backend = "setuptools.build_meta"
0060
0061 [project]
0062 name = "llmctg"
0063 version = "1.0.0"
0064 description = "LLM Chinese Text Generator"
0065 readme = "README.md"
0066 requires-python = ">=3.10"
0067 license = {text = "Proprietary"}
0068 dependencies = []
0069
0070 [project.optional-dependencies]
0071 llm = [
0072     "torch==2.6.0",
0073     "transformers>=4.51,<5",
0074     "peft>=0.10,<1",
0075     "datasets>=2.18,<5",
0076     "accelerate>=0.26,<2",
0077 ]
0078 dev = ["pytest>=8.0"]
0079
0080 [project.scripts]
0081 llmctg = "llmctg.cli:main"
0082
0083 [tool.setuptools]
0084 package-dir = {"" = "src"}
0085
0086 [tool.setuptools.packages.find]
0087 where = ["src"]
0088
0089 [tool.pytest.ini_options]
0090 pythonpath = ["src"]
0091 testpaths = ["tests"]
0092
0093 ===== 文件开始 scripts/export_docs_markdown.py =====
0094 """Export the generated manual and current source tree as Markdown deliverables."""
0095
0096 from __future__ import annotations
0097
0098 from pathlib import Path
```

源程序第 3 页 scripts/export_docs_markdown.py

```
0007 from docx import Document
0008 from docx.document import Document as DocumentObject
0009 from docx.xml.table import CT_Tbl
0010 from docx.xml.text.paragraph import CT_P
0011 from docx.table import Table
0012 from docx.text.paragraph import Paragraph
0013
0014 from build_copyright_docs import DELIVERY, SOFTWARE_NAME, VERSION, source_files
0015
0016 ROOT = Path(__file__).resolve().parents[1]
0017
0018
0019 def iter_blocks(parent):
0020     parent_element = parent.element.body if isinstance(parent, DocumentObject) else parent._tc
0021     for child in parent_element.iterchildren():
0022         if isinstance(child, CT_P):
0023             yield Paragraph(child, parent)
0024         elif isinstance(child, CT_Tbl):
0025             yield Table(child, parent)
0026
0027
0028 def escape_cell(text: str) -> str:
0029     return "<br>".join(part.strip() for part in text.splitlines()).replace("|", "\\|")
0030
0031
0032 def paragraph_is_code(paragraph: Paragraph) -> bool:
0033     visible_runs = [run for run in paragraph.runs if run.text]
0034     return bool(visible_runs) and all(run.font.name == "Consolas" for run in visible_runs)
0035
0036
0037 def manual_to_markdown(source: Path, target: Path) -> None:
0038     document = Document(source)
0039     output: list[str] = []
0040     code_lines: list[str] = []
0041
0042     def flush_code() -> None:
0043         if code_lines:
0044             output.extend(["```bash", *code_lines, "```", ""])
0045             code_lines.clear()
0046
0047     for block in iter_blocks(document):
0048         if isinstance(block, Paragraph):
0049             text = block.text.strip()
0050             if not text:
0051                 flush_code()
0052                 continue
0053             if paragraph_is_code(block):
0054                 code_lines.append(block.text)
0055                 continue
0056             flush_code()
```

源程序第 4 页 scripts/export_docs_markdown.py

```
0057         style = block.style.name if block.style else ""
0058         if style == "Title":
0059             output.extend([f"# {text}", ""])
0060         elif style.startswith("Heading "):
0061             level = int(style.rsplit(" ", 1)[1])
0062             output.extend([f"{'#' * (level + 1)} {text}", ""])
0063         elif style.startswith("List Bullet"):
0064             output.extend([f"- {text}", ""])
0065         else:
0066             output.extend([text, ""])
0067     else:
0068         flush_code()
0069         rows = [[escape_cell(cell.text) for cell in row.cells] for row in block.rows]
0070         if not rows:
0071             continue
0072         output.append("| " + " | ".join(rows[0]) + " |")
0073         output.append("| " + " | ".join("---" for _ in rows[0]) + " |")
0074         for row in rows[1:]:
0075             output.append("| " + " | ".join(row) + " |")
0076         output.append("")
0077     flush_code()
0078     target.write_text("\n".join(output).rstrip() + "\n", encoding="utf-8")
0079
0080
0081 def source_to_markdown(target: Path) -> None:
0082     files = source_files()
0083     total_lines = 0
0084     sections: list[str] = [
0085         f"# {SOFTWARE_NAME}{VERSION}源程序鉴别材料",
0086         "",
0087         "本文档收录当前 LLMCTG 目录中的核心程序、命令脚本、配置和自动化测试源码。",
0088         "生成文档的构建脚本不纳入鉴别材料。",
0089         "",
0090         "## 文件清单",
0091         "",
0092     ]
0093     for path in files:
0094         relative = path.relative_to(ROOT).as_posix()
0095         sections.append(f"- `{relative}`")
0096     for path in files:
0097         relative = path.relative_to(ROOT).as_posix()
0098         content = path.read_text(encoding="utf-8-sig").replace("\r\n", "\n").replace("\r", "\n")
0099         lines = content.splitlines()
0100         total_lines += len(lines)
0101         language = {
0102             ".py": "python", ".ps1": "powershell", ".sh": "bash",
0103             ".toml": "toml", ".json": "json",
0104         }.get(path.suffix.lower(), "text")
0105         sections.extend([
0106             "",
```

源程序第 5 页 scripts/export_docs_markdown.py

```
0107         f"## {relative}",
0108         "",
0109         f"```{language}",
0110         content.rstrip(),
0111         "```",
0112     ])
0113     sections[4:4] = [f"共收录 {len(files)} 个文件, 约 {total_lines} 行源代码。", ""]
0114     target.write_text("\n".join(sections).rstrip() + "\n", encoding="utf-8")
0115
0116
0117 def main() -> None:
0118     DELIVERY.mkdir(parents=True, exist_ok=True)
0119     manual_docx = DELIVERY / f"{SOFTWARE_NAME}{VERSION}用户操作手册.docx"
0120     manual_md = DELIVERY / f"{SOFTWARE_NAME}{VERSION}用户操作手册.md"
0121     source_md = DELIVERY / f"{SOFTWARE_NAME}{VERSION}源程序鉴别材料.md"
0122     manual_to_markdown(manual_docx, manual_md)
0123     source_to_markdown(source_md)
0124     print(manual_md)
0125     print(source_md)
0126
0127
0128 if __name__ == "__main__":
0129     main()
0130
0131 ===== 文件开始 scripts/one_click_test.py =====
0132
0133 0001 #!/usr/bin/env python3
0134 0002 """LLM 中文文本生成系统一键功能测试。
0135 0003
0136 0004 默认使用项目在 Linux 服务器上的预设路径。每项测试执行前都会显示测试功能和
0137 0005 预期结果；所有产物写入带时间戳的新目录，不覆盖既有测试结果。
0138 0006 """
0139 0007
0140 0008 from __future__ import annotations
0141 0009
0142 0010 import argparse
0143 0011 import json
0144 0012 import os
0145 0013 import subprocess
0146 0014 import sys
0147 0015 from datetime import datetime
0148 0016 from pathlib import Path
0149 0017 from typing import Callable
0150 0018
0151 0019
0152 0020 DEFAULT_PROJECT_ROOT = Path("/home/xxx/workplace/xxx/llmctg")
0153 0021 DEFAULT_MODEL_PATH = Path("/home/xxx/workplace/xxx/Qwen3-1.7B")
0154 0022
0155 0023
0156 0024 class TestFailure(RuntimeError):
0157 0025     pass
```

源程序第 6 页 scripts/one_click_test.py

```
0026
0027
0028 class Runner:
0029     def __init__(self, project_root: Path, model_path: Path, output_root: Path, device: str):
0030         self.project_root = project_root
0031         self.model_path = model_path
0032         self.output_root = output_root
0033         self.device = device
0034         self.results: list[dict[str, str]] = []
0035         self.env = os.environ.copy()
0036         if device.startswith("cuda:"):
0037             # 对外只暴露选中的物理卡；进程内统一使用 cuda:0，避免 Trainer 自动多卡。
0038             self.env["CUDA_VISIBLE_DEVICES"] = device.split(":")[1]
0039             self.process_device = "cuda:0"
0040         else:
0041             self.process_device = device
0042
0043     def announce(self, number: int, name: str, expected: str) -> None:
0044         print("\n" + "=" * 78)
0045         print(f"测试 {number}: {name}")
0046         print(f"运行功能: {name}")
0047         print(f"预期结果: {expected}")
0048         print("=" * 78, flush=True)
0049
0050     def command(self, *arguments: str, accepted_codes: set[int] | None = None) -> None:
0051         command = [sys.executable, "-m", "llmctg", *map(str, arguments)]
0052         print("执行命令:", " ".join(command), flush=True)
0053         completed = subprocess.run(
0054             command,
0055             cwd=self.project_root,
0056             env=self.env,
0057             text=True,
0058         )
0059         accepted = accepted_codes or {}
0060         if completed.returncode not in accepted:
0061             raise TestFailure(f"命令退出码为 {completed.returncode}，预期为 {sorted(accepted)}")
0062
0063     @staticmethod
0064     def require(path: Path, description: str) -> None:
0065         if not path.exists():
0066             raise TestFailure(f"未生成{description}: {path}")
0067
0068     @staticmethod
0069     def require_json(path: Path) -> dict:
0070         Runner.require(path, "JSON 文件")
0071         with path.open("r", encoding="utf-8") as stream:
0072             return json.load(stream)
0073
0074     @staticmethod
0075     def require_jsonl(path: Path, minimum: int = 1) -> list[dict]:
```

源程序第 7 页 scripts/one_click_test.py

```
0076         Runner.require(path, "JSONL 文件")
0077         with path.open("r", encoding="utf-8") as stream:
0078             records = [json.loads(line) for line in stream if line.strip()]
0079         if len(records) < minimum:
0080             raise TestFailure(f"{path} 只有 {len(records)} 条记录, 预期至少 {minimum} 条")
0081         return records
0082
0083     def execute(self, number: int, name: str, expected: str, function: Callable[[], None]) -> None:
0084         self.announce(number, name, expected)
0085         try:
0086             function()
0087         except Exception as error:
0088             self.results.append({"test": name, "status": "FAIL", "detail": str(error)})
0089             print(f"[失败] {error}", flush=True)
0090         else:
0091             self.results.append({"test": name, "status": "PASS", "detail": expected})
0092             print(f"[通过] {expected}", flush=True)
0093
0094     def test_version(self) -> None:
0095         self.command("--version")
0096
0097     def test_init_config(self) -> None:
0098         target = self.output_root / "llmctg.json"
0099         self.command("init-config", "--output", target)
0100         payload = self.require_json(target)
0101         if "generation" not in payload or "evaluation" not in payload:
0102             raise TestFailure("配置文件缺少 generation 或 evaluation")
0103
0104     def test_prepare_data(self) -> None:
0105         source = self.project_root / "test_data" / "llmctg_smoke_train.txt"
0106         target = self.output_root / "prepared_train.jsonl"
0107         self.command("prepare-data", "--input", source, "--output", target)
0108         self.require_jsonl(target, minimum=12)
0109
0110     def test_template_generate(self) -> None:
0111         target = self.output_root / "template_generation.jsonl"
0112         self.command(
0113             "generate", "--backend", "template", "--topic", "城市河流",
0114             "--level", "junior", "--keywords", "河流,生态,保护",
0115             "--count", "1", "--seed", "2026", "--output", target,
0116         )
0117         records = self.require_jsonl(target)
0118         if records[0].get("backend") != "template":
0119             raise TestFailure("生成结果的 backend 不是 template")
0120
0121     def test_evaluate(self) -> None:
0122         target = self.output_root / "evaluation.json"
0123         text = (
0124             "城市河流为植物和动物提供生活空间, 也能调节局部气候。"
0125             "保护河流需要减少污水排放、修复河岸植被, 并鼓励居民节约用水。"
```

源程序第 8 页 scripts/one_click_test.py

```
0126         )
0127         self.command(
0128             "evaluate", "--text", text, "--level", "junior", "--output", target,
0129             accepted_codes={0, 1},
0130         )
0131         payload = self.require_json(target)
0132         if "metrics" not in payload or "statistics" not in payload:
0133             raise TestFailure("评估结果缺少 metrics 或 statistics")
0134
0135     def test_batch(self) -> None:
0136         output = self.output_root / "batch"
0137         self.command(
0138             "batch", "--backend", "template",
0139             "--input", self.project_root / "examples" / "tasks.jsonl",
0140             "--output", output, "--no-resume",
0141         )
0142         self.require_jsonl(output / "articles.jsonl", minimum=3)
0143         self.require(output / "summary.json", "批处理摘要")
0144         self.require(output / "summary.csv", "批处理 CSV")
0145         self.require(output / "report.html", "批处理 HTML 报告")
0146
0147     def test_demo(self) -> None:
0148         output = self.output_root / "demo"
0149         self.command("demo", "--output", output)
0150         self.require_jsonl(output / "articles.jsonl", minimum=3)
0151         self.require(output / "report.html", "演示 HTML 报告")
0152
0153     def test_base_model_generate(self) -> None:
0154         target = self.output_root / "qwen3_base_generation.jsonl"
0155         self.command(
0156             "generate", "--backend", "transformers", "--model-path", self.model_path,
0157             "--device", self.process_device, "--topic", "城市河流与生态保护",
0158             "--level", "junior",
0159             "--instruction", "说明城市河流的生态价值、主要污染来源和保护方法",
0160             "--keywords", "河流,生态,污染,保护", "--count", "1", "--seed", "2026",
0161             "--output", target,
0162         )
0163         records = self.require_jsonl(target)
0164         if not records[0].get("content", "").strip():
0165             raise TestFailure("本地模型生成正文为空")
0166
0167     def test_train(self) -> None:
0168         dataset = self.output_root / "prepared_train.jsonl"
0169         adapter = self.output_root / "qwen3-1.7b-lora-smoke"
0170         self.command(
0171             "train", "--model-path", self.model_path, "--dataset", dataset,
0172             "--output", adapter, "--epochs", "1", "--batch-size", "1",
0173             "--max-length", "256", "--device", self.process_device,
0174         )
0175         self.require(adapter / "adapter_config.json", "LoRA 配置")
```

源程序第 9 页 scripts/one_click_test.py

```
0176         self.require(adapter / "adapter_model.safetensors", "LoRA 权重")
0177
0178     def test_lora_generate(self) -> None:
0179         adapter = self.output_root / "qwen3-1.7b-lora-smoke"
0180         target = self.output_root / "qwen3_lora_generation.jsonl"
0181         self.command(
0182             "generate", "--backend", "transformers", "--model-path", self.model_path,
0183             "--lora-path", adapter, "--device", self.process_device,
0184             "--topic", "城市河流与生态保护", "--level", "junior",
0185             "--instruction", "说明城市河流的生态价值、主要污染来源和保护方法",
0186             "--keywords", "河流,生态,污染,保护", "--count", "1", "--seed", "2026",
0187             "--output", target,
0188         )
0189         records = self.require_jsonl(target)
0190         if not records[0].get("content", "").strip():
0191             raise TestFailure("LoRA 模型生成正文为空")
0192
0193     def write_summary(self) -> Path:
0194         summary = self.output_root / "test_summary.json"
0195         summary.write_text(
0196             json.dumps(self.results, ensure_ascii=False, indent=2),
0197             encoding="utf-8",
0198         )
0199         return summary
0200
0201
0202     def parse_arguments() -> argparse.Namespace:
0203         parser = argparse.ArgumentParser(description="LLM 中文文本生成系统一键功能测试")
0204         parser.add_argument("--project-root", type=Path, default=DEFAULT_PROJECT_ROOT)
0205         parser.add_argument("--model-path", type=Path, default=DEFAULT_MODEL_PATH)
0206         parser.add_argument("--device", default="cuda:0", help="cpu、cuda 或 cuda:N")
0207         parser.add_argument("--skip-llm", action="store_true", help="跳过本地模型、训练及 LoRA 测试")
0208         return parser.parse_args()
0209
0210
0211     def main() -> int:
0212         args = parse_arguments()
0213         project_root = args.project_root.resolve()
0214         model_path = args.model_path.resolve()
0215         stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
0216         output_root = project_root / "test_output" / f"one_click_{stamp}"
0217         output_root.mkdir(parents=True, exist_ok=False)
0218
0219         print("LLM 中文文本生成系统——一键功能测试")
0220         print(f"项目路径: {project_root}")
0221         print(f"模型路径: {model_path}")
0222         print(f"输出路径: {output_root}")
0223         print(f"计算设备: {args.device}")
0224
0225         runner = Runner(project_root, model_path, output_root, args.device)
```

源程序第 10 页 scripts/one_click_test.py

```
0226     tests: list[tuple[str, str, Callable[[], None]]] = [
0227         ("版本与命令入口", "输出 llmctg 版本号, 退出码为 0。", runner.test_version),
0228         ("默认配置生成", "生成可解析且包含 generation、evaluation 的 JSON 配置。", runner.test_init_config),
0229         ("训练数据转换", "把标注文本转换成至少 12 条 JSONL 训练样本。", runner.test_prepare_data),
0230         ("模板文本生成", "生成 1 条正文且 backend 为 template。", runner.test_template_generate),
0231         ("文本质量评估", "生成包含统计指标和质量指标的 JSON; 是否及格不影响功能判定。", runner.test_evaluate),
0232         ("批量任务处理", "处理 3 条任务并生成 JSONL、JSON、CSV 和 HTML 报告。", runner.test_batch),
0233         ("完整离线演示", "生成 3 篇示例文章及汇总报告。", runner.test_demo),
0234     ]
0235     if not args.skip_llm:
0236         tests.extend([
0237             ("本地基础模型生成", "Qwen3 基础模型生成一条非空中文文本。", runner.test_base_model_generate),
0238             ("LoRA 冒烟微调", "完成 1 轮训练并保存 adapter_config.json 和适配器权重。", runner.test_train),
0239             ("LoRA 适配器生成", "加载刚训练的适配器并生成一条非空中文文本。", runner.test_lora_generate),
0240         ])
0241
0242     for number, (name, expected, function) in enumerate(tests, 1):
0243         runner.execute(number, name, expected, function)
0244
0245     summary = runner.write_summary()
0246     failed = [item for item in runner.results if item["status"] == "FAIL"]
0247     print("\n" + "=" * 78)
0248     print(f"测试完成: 通过 {len(runner.results) - len(failed)}, 失败 {len(failed)}")
0249     print(f"汇总文件: {summary}")
0250     print(f"全部产物: {output_root}")
0251     if failed:
0252         print("失败项目: " + "、".join(item["test"] for item in failed))
0253         return 1
0254     print("预期结果: 所有测试均显示 PASS, LoRA 适配器及生成结果均已保存。")
0255     return 0
0256
0257
0258 if __name__ == "__main__":
0259     raise SystemExit(main())
0260
0261
0262 ===== 文件开始 scripts/run_demo.ps1 =====
0001 $ErrorActionPreference = "Stop"
0002 $projectRoot = Split-Path -Parent $PSScriptRoot
0003 $env:PYTHONPATH = Join-Path $projectRoot "src"
0004 python -m llmctg demo --output (Join-Path $projectRoot "output\demo")
0005
0263 ===== 文件开始 scripts/run_demo.sh =====
0001 #!/usr/bin/env bash
0002 set -euo pipefail
0003 PROJECT_ROOT="$(cd "$(dirname "$0")/.." && pwd)"
0004 export PYTHONPATH="$PROJECT_ROOT/src"
0005 python -m llmctg demo --output "$PROJECT_ROOT/output/demo"
0006
0264 ===== 文件开始 scripts/run_tests.ps1 =====
0001 $ErrorActionPreference = "Stop"
```

源程序第 11 页 scripts/run_tests.ps1

```
0002 $projectRoot = Split-Path -Parent $PSScriptRoot
0003 $env:PYTHONPATH = Join-Path $projectRoot "src"
0004 python -m pytest (Join-Path $projectRoot "tests") -q
0005
===== 文件开始 src/llmctg/__init__.py =====
0001 """LLM 中文文本生成系统。"""
0002
0003 from .models import ReadingLevel, GenerationRequest, GeneratedArticle, EvaluationResult
0004
0005 __all__ = ["ReadingLevel", "GenerationRequest", "GeneratedArticle", "EvaluationResult"]
0006 __version__ = "1.0.0"
0007
===== 文件开始 src/llmctg/__main__.py =====
0001 from .cli import main
0002
0003 if __name__ == "__main__":
0004     raise SystemExit(main())
0005
===== 文件开始 src/llmctg/cli.py =====
0001 from __future__ import annotations
0002
0003 import argparse
0004 import json
0005 import sys
0006 from pathlib import Path
0007 from typing import Sequence
0008
0009 from . import __version__
0010 from .config import AppConfig
0011 from .dataset import convert_labeled_text
0012 from .io_utils import append_jsonl, write_json
0013 from .logging_utils import configure_logging
0014 from .models import GenerationRequest, ReadingLevel
0015 from .pipeline import LLCTGApplication
0016 from .reporting import build_summary, write_html_report, write_summary_csv
0017 from .text_utils import read_text
0018 from .training import FineTuneOptions, LoraTrainer
0019
0020
0021 def build_parser() -> argparse.ArgumentParser:
0022     parser = argparse.ArgumentParser(
0023         prog="llmctg",
0024         description="LLM 中文文本生成系统",
0025     )
0026     parser.add_argument("--version", action="version", version=f"%(prog)s {__version__}")
0027     parser.add_argument("--config", help="JSON 配置文件路径")
0028     parser.add_argument("--log-level", choices=["DEBUG", "INFO", "WARNING", "ERROR"])
0029     subparsers = parser.add_subparsers(dest="command", required=True)
0030
0031     generate = subparsers.add_parser("generate", help="生成并评估阅读材料")
```

源程序第 12 页 src/llmctg/cli.py

```
0032     _add_generation_arguments(generate)
0033     generate.add_argument("--output", help="输出 JSONL 文件, 省略时打印到终端")
0034
0035     evaluate = subparsers.add_parser("evaluate", help="评估已有文本")
0036     evaluate_source = evaluate.add_mutually_exclusive_group(required=True)
0037     evaluate_source.add_argument("--text", help="直接传入文本")
0038     evaluate_source.add_argument("--input", help="UTF-8 文本文件")
0039     evaluate.add_argument("--level", default="junior", help="目标等级")
0040     evaluate.add_argument("--output", help="输出 JSON 文件")
0041
0042     batch = subparsers.add_parser("batch", help="执行 JSONL 批量任务")
0043     batch.add_argument("--input", required=True, help="任务 JSONL 文件")
0044     batch.add_argument("--output", required=True, help="结果目录")
0045     batch.add_argument("--no-resume", action="store_true", help="不读取已有结果的任务编号")
0046     _add_backend_arguments(batch)
0047
0048     demo = subparsers.add_parser("demo", help="运行无需模型的完整演示")
0049     demo.add_argument("--output", default="output/demo", help="演示输出目录")
0050
0051     init = subparsers.add_parser("init-config", help="生成默认配置文件")
0052     init.add_argument("--output", default="llmctg.json", help="配置文件路径")
0053
0054     prepare = subparsers.add_parser("prepare-data", help="把制表符标注文本转换为训练 JSONL")
0055     prepare.add_argument("--input", required=True, help="每行格式为 正文 TAB 等级")
0056     prepare.add_argument("--output", required=True, help="训练 JSONL 输出路径")
0057
0058     train = subparsers.add_parser("train", help="使用本地模型执行 LoRA 微调")
0059     train.add_argument("--model-path", required=True)
0060     train.add_argument("--dataset", required=True)
0061     train.add_argument("--output", required=True)
0062     train.add_argument("--epochs", type=float, default=3.0)
0063     train.add_argument("--batch-size", type=int, default=1)
0064     train.add_argument("--max-length", type=int, default=1024)
0065     train.add_argument("--device", default="auto")
0066     return parser
0067
0068
0069 def _add_generation_arguments(parser: argparse.ArgumentParser) -> None:
0070     parser.add_argument("--topic", required=True, help="文章主题")
0071     parser.add_argument("--level", default="junior", help="primary、junior、senior 或中文等级")
0072     parser.add_argument("--instruction", default="", help="额外创作要求")
0073     parser.add_argument("--keywords", default="", help="用逗号分隔的关键词")
0074     parser.add_argument("--count", type=int, default=1, help="生成数量, 1 到 20")
0075     parser.add_argument("--seed", type=int, help="随机种子")
0076     _add_backend_arguments(parser)
0077
0078
0079 def _add_backend_arguments(parser: argparse.ArgumentParser) -> None:
0080     parser.add_argument("--backend", choices=["template", "transformers"], help="生成后端")
0081     parser.add_argument("--model-path", help="本地模型目录")
```

源程序第 13 页 src/llmctg/cli.py

```
0082     parser.add_argument("--lora-path", help="LoRA 适配器目录, 仅用于 transformers 后端")
0083     parser.add_argument("--device", help="auto、cpu、cuda 或 cuda:N")
0084
0085
0086 def main(argv: Sequence[str] | None = None) -> int:
0087     parser = build_parser()
0088     args = parser.parse_args(argv)
0089     try:
0090         config = AppConfig.load(args.config)
0091         if args.log_level:
0092             config.log_level = args.log_level
0093         _apply_backend_arguments(config, args)
0094         configure_logging(config.log_level)
0095         if args.command == "init-config":
0096             path = config.save(args.output)
0097             print(f"已生成配置文件: {path}")
0098             return 0
0099         if args.command == "prepare-data":
0100             count = convert_labeled_text(args.input, args.output)
0101             print(f"数据转换完成, 共 {count} 条: {args.output}")
0102             return 0
0103         if args.command == "train":
0104             result = LoraTrainer(FineTuneOptions(
0105                 model_path=args.model_path,
0106                 dataset_path=args.dataset,
0107                 output_dir=args.output,
0108                 epochs=args.epochs,
0109                 batch_size=args.batch_size,
0110                 max_length=args.max_length,
0111                 device=args.device,
0112             )).train()
0113             print(json.dumps(result, ensure_ascii=False, indent=2))
0114             return 0
0115         if args.command == "demo":
0116             return _run_demo(config, args)
0117         with LLMCTGApplication(config) as studio:
0118             if args.command == "generate":
0119                 return _run_generate(studio, args)
0120             if args.command == "evaluate":
0121                 return _run_evaluate(studio, args)
0122             if args.command == "batch":
0123                 summary = studio.batch(args.input, args.output, resume=not args.no_resume)
0124                 print(json.dumps(summary.to_dict(), ensure_ascii=False, indent=2))
0125                 return 0 if summary.processing_errors == 0 else 2
0126         parser.error("未知命令")
0127     except (ValueError, FileNotFoundError, RuntimeError) as error:
0128         print(f"错误: {error}", file=sys.stderr)
0129         return 2
0130     return 0
0131
```

```

0132
0133 def _apply_backend_arguments(config: AppConfig, args: argparse.Namespace) -> None:
0134     for name in ("backend", "model_path", "lora_path", "device"):
0135         value = getattr(args, name, None)
0136         if value:
0137             setattr(config.generation, name, value)
0138
0139
0140 def _run_generate(studio: LLMCTGApplication, args: argparse.Namespace) -> int:
0141     keywords = [item.strip() for item in args.keywords.replace(", ", ",").split(",") if item.strip()]
0142     request = GenerationRequest(
0143         topic=args.topic,
0144         level=ReadingLevel.parse(args.level),
0145         instruction=args.instruction,
0146         keywords=keywords,
0147         count=args.count,
0148         seed=args.seed,
0149     )
0150     articles = studio.generate(request)
0151     for article in articles:
0152         if args.output:
0153             append_jsonl(args.output, article.to_dict())
0154         else:
0155             print(json.dumps(article.to_dict(), ensure_ascii=False, indent=2))
0156     return 0
0157
0158
0159 def _run_evaluate(studio: LLMCTGApplication, args: argparse.Namespace) -> int:
0160     text = args.text if args.text is not None else read_text(args.input)
0161     result = studio.evaluate(text, ReadingLevel.parse(args.level))
0162     payload = result.to_dict()
0163     if args.output:
0164         write_json(args.output, payload)
0165         print(f"评估结果已保存: {args.output}")
0166     else:
0167         print(json.dumps(payload, ensure_ascii=False, indent=2))
0168     return 0 if result.passed else 1
0169
0170
0171 def _run_demo(config: AppConfig, args: argparse.Namespace) -> int:
0172     config.generation.backend = "template"
0173     output = Path(args.output)
0174     output.mkdir(parents=True, exist_ok=True)
0175     articles = []
0176     with LLMCTGApplication(config) as studio:
0177         for index, (topic, level) in enumerate([
0178             ("四季的变化", ReadingLevel.PRIMARY),
0179             ("城市里的河流", ReadingLevel.JUNIOR),
0180             ("人工智能与学习", ReadingLevel.SENIOR),
0181         ]):

```

源程序第 15 页 src/llmctg/cli.py

```
0182         request = GenerationRequest(topic=topic, level=level, count=1, seed=2026 + index)
0183         generated = studio.generate(request)
0184         articles.extend(generated)
0185         append_jsonl(output / "articles.jsonl", generated[0].to_dict())
0186         summary = build_summary(articles)
0187         write_json(output / "summary.json", summary.to_dict())
0188         write_summary_csv(output / "summary.csv", articles)
0189         write_html_report(output / "report.html", articles, summary)
0190         print(f"演示完成, 结果目录: {output.resolve()}")
0191         return 0
0192
0193
0194 if __name__ == "__main__":
0195     raise SystemExit(main())
0196
===== 文件开始 src/llmctg/config.py =====
0001 from __future__ import annotations
0002
0003 import json
0004 from dataclasses import asdict, dataclass, field
0005 from pathlib import Path
0006 from typing import Any
0007
0008
0009 @dataclass(slots=True)
0010 class GenerationConfig:
0011     backend: str = "template"
0012     model_path: str = ""
0013     device: str = "auto"
0014     lora_path: str = ""
0015     temperature: float = 0.9
0016     top_p: float = 0.9
0017     top_k: int = 50
0018     repetition_penalty: float = 1.1
0019     max_new_tokens_primary: int = 380
0020     max_new_tokens_junior: int = 800
0021     max_new_tokens_senior: int = 1200
0022
0023
0024 @dataclass(slots=True)
0025 class EvaluationConfig:
0026     pass_score: float = 60.0
0027     excellent_score: float = 85.0
0028     good_score: float = 70.0
0029     primary_sentence_target: float = 16.0
0030     junior_sentence_target: float = 24.0
0031     senior_sentence_target: float = 34.0
0032     repetition_warning: float = 0.18
0033     english_warning: float = 0.15
0034
```

```

0035
0036 @dataclass(slots=True)
0037 class AppConfig:
0038     generation: GenerationConfig = field(default_factory=GenerationConfig)
0039     evaluation: EvaluationConfig = field(default_factory=EvaluationConfig)
0040     log_level: str = "INFO"
0041     output_encoding: str = "utf-8"
0042
0043     @classmethod
0044     def load(cls, path: str | Path | None = None) -> "AppConfig":
0045         if not path:
0046             return cls()
0047         config_path = Path(path)
0048         if not config_path.exists():
0049             raise FileNotFoundError(f"配置文件不存在: {config_path}")
0050         with config_path.open("r", encoding="utf-8") as stream:
0051             raw = json.load(stream)
0052         return cls(
0053             generation=GenerationConfig(**raw.get("generation", {})),
0054             evaluation=EvaluationConfig(**raw.get("evaluation", {})),
0055             log_level=raw.get("log_level", "INFO"),
0056             output_encoding=raw.get("output_encoding", "utf-8"),
0057         )
0058
0059     def save(self, path: str | Path) -> Path:
0060         target = Path(path)
0061         target.parent.mkdir(parents=True, exist_ok=True)
0062         with target.open("w", encoding="utf-8") as stream:
0063             json.dump(asdict(self), stream, ensure_ascii=False, indent=2)
0064         return target
0065
0066     def merge(self, values: dict[str, Any]) -> "AppConfig":
0067         for section, section_values in values.items():
0068             target = getattr(self, section, None)
0069             if target is None or not isinstance(section_values, dict):
0070                 continue
0071             for name, value in section_values.items():
0072                 if hasattr(target, name) and value is not None:
0073                     setattr(target, name, value)
0074         return self
0075
0076 ===== 文件开始 src/llmctg/dataset.py =====
0001 from __future__ import annotations
0002
0003 import json
0004 from dataclasses import dataclass
0005 from pathlib import Path
0006 from typing import Iterator
0007
0008 from .models import ReadingLevel

```

```

0009
0010
0011 @dataclass(slots=True)
0012 class TrainingSample:
0013     instruction: str
0014     input: str
0015     output: str
0016     level: ReadingLevel
0017
0018     def to_dict(self) -> dict[str, str]:
0019         return {
0020             "instruction": self.instruction,
0021             "input": self.input,
0022             "output": self.output,
0023             "level": self.level.value,
0024         }
0025
0026
0027 def parse_labeled_text(path: str | Path) -> Iterator[TrainingSample]:
0028     source = Path(path)
0029     with source.open("r", encoding="utf-8-sig") as stream:
0030         for line_number, line in enumerate(stream, 1):
0031             stripped = line.strip()
0032             if not stripped:
0033                 continue
0034             try:
0035                 text, raw_level = stripped.rsplit("\t", 1)
0036             except ValueError as error:
0037                 raise ValueError(f"{source} 第 {line_number} 行缺少制表符和等级标签") from error
0038             level = ReadingLevel.parse(raw_level)
0039             text = text.strip()
0040             if not text:
0041                 raise ValueError(f"{source} 第 {line_number} 行正文为空")
0042             yield TrainingSample(
0043                 instruction=f"请生成一段适合{level.chinese_name}学生阅读的中文内容。",
0044                 input="",
0045                 output=text,
0046                 level=level,
0047             )
0048
0049
0050 def convert_labeled_text(input_path: str | Path, output_path: str | Path) -> int:
0051     samples = list(parse_labeled_text(input_path))
0052     target = Path(output_path)
0053     target.parent.mkdir(parents=True, exist_ok=True)
0054     with target.open("w", encoding="utf-8", newline="\n") as stream:
0055         for sample in samples:
0056             stream.write(json.dumps(sample.to_dict(), ensure_ascii=False) + "\n")
0057     return len(samples)
0058

```

源程序第 18 页 src/llmctg/evaluator.py

```
==== 文件开始 src/llmctg/evaluator.py ====
0001 from __future__ import annotations
0002
0003 import math
0004 import re
0005 from collections import Counter
0006
0007 from .config import EvaluationConfig
0008 from .models import EvaluationResult, QualityMetrics, ReadingLevel, TextStatistics
0009 from .text_utils import chinese_characters, normalize_text, repetition_rate, split_paragraphs, split_sentences
0010
0011 COMMON_CHARACTERS = set(
0012     "的一是在不了有和人这中大为上个国我以要他时来用们生到作地于出就分对成会可主发年动同工也能下过予说产种面而方后多定行学法所民得经十三之进着等部度家电力里
0013     如水化高自二理起小物现实加量都两体制机当使点从业本去把性好应开它合还因由某些然前外天政四日那社义事平形相全表间样与关各重新线内数正心反你明看原又么利比或但质气第向
0014     道命此变条只没结解问意建月公无系军很情者最立代想已通并提直题党程展五果料象员革位入常文总次品式活设及管特件长求老头基资边流路级少图山统接知较将组见计别她手角期根论
0015     运农指几九区强放决西被干做必战先回则任取据处理世车更斗百给名真论条形"
0016 )
0017 PUNCTUATION = set(",。!?!;:、,。!?!;:‘’""()《》〈〉-_-\\n")
0018
0019 class ReadabilityEvaluator:
0020     def __init__(self, config: EvaluationConfig | None = None):
0021         self.config = config or EvaluationConfig()
0022
0023     def evaluate(self, text: str, target_level: ReadingLevel | str | int) -> EvaluationResult:
0024         target = ReadingLevel.parse(target_level)
0025         normalized = normalize_text(text)
0026         statistics = self._statistics(normalized)
0027         predicted = self._predict_level(statistics, normalized)
0028         metrics = self._metrics(normalized, target, predicted, statistics)
0029         warnings, suggestions = self._advice(normalized, target, statistics, metrics)
0030         grade = self._grade(metrics.overall_score)
0031         passed = metrics.overall_score >= self.config.pass_score and len(normalized) > 0
0032         return EvaluationResult(
0033             target_level=target,
0034             predicted_level=predicted,
0035             quality_grade=grade,
0036             passed=passed,
0037             statistics=statistics,
0038             metrics=metrics,
0039             warnings=warnings,
0040             suggestions=suggestions,
0041         )
0042
0043     def _statistics(self, text: str) -> TextStatistics:
0044         chars = chinese_characters(text)
0045         sentences = split_sentences(text)
0046         paragraphs = split_paragraphs(text)
0047         lengths = [len(chinese_characters(item)) for item in sentences]
0048         unique_ratio = len(set(chars)) / len(chars) if chars else 0.0
0049         punctuation_count = sum(1 for char in text if char in PUNCTUATION)
0050         return TextStatistics(
0051             characters=len(text.replace("\\n", "")),
```

```

0050         chinese_characters=len(chars),
0051         sentences=len(sentences),
0052         paragraphs=len(paragraphs),
0053         average_sentence_length=sum(lengths) / len(lengths) if lengths else 0.0,
0054         max_sentence_length=max(lengths, default=0),
0055         unique_character_ratio=round(unique_ratio, 4),
0056         punctuation_ratio=round(punctuation_count / len(text), 4) if text else 0.0,
0057     )
0058
0059     def _predict_level(self, stats: TextStatistics, text: str) -> ReadingLevel:
0060         rare_ratio = self._uncommon_ratio(text)
0061         complexity = stats.average_sentence_length + rare_ratio * 55 + max(0, stats.max_sentence_length - 30) * 0.12
0062         if complexity < 20:
0063             return ReadingLevel.PRIMARY
0064         if complexity < 31:
0065             return ReadingLevel.JUNIOR
0066         return ReadingLevel.SENIOR
0067
0068     def _metrics(
0069         self,
0070         text: str,
0071         target: ReadingLevel,
0072         predicted: ReadingLevel,
0073         stats: TextStatistics,
0074     ) -> QualityMetrics:
0075         repeat = repetition_rate(text, size=4)
0076         english_count = sum(1 for char in text if "a" <= char.lower() <= "z")
0077         english = english_count / max(1, len(text))
0078         uncommon = self._uncommon_ratio(text)
0079         structure = self._structure_score(text, stats)
0080         length = self._length_score(target, stats.chinese_characters)
0081         distance = abs(predicted.numeric_label - target.numeric_label)
0082         level_score = max(0.0, 100.0 - distance * 38.0)
0083         fluency_score = max(0.0, 100.0 - repeat * 260.0 - english * 100.0 - uncommon * 65.0)
0084         overall = structure * 0.20 + length * 0.20 + level_score * 0.30 + fluency_score * 0.30
0085         return QualityMetrics(
0086             repetition_rate=round(repeat, 4),
0087             english_ratio=round(english, 4),
0088             uncommon_character_ratio=round(uncommon, 4),
0089             structure_score=round(structure, 2),
0090             length_score=round(length, 2),
0091             level_score=round(level_score, 2),
0092             overall_score=round(overall, 2),
0093         )
0094
0095     @staticmethod
0096     def _uncommon_ratio(text: str) -> float:
0097         chars = chinese_characters(text)
0098         if not chars:
0099             return 0.0

```

```

0100     return sum(1 for char in chars if char not in COMMON_CHARACTERS) / len(chars)
0101
0102     @staticmethod
0103     def _structure_score(text: str, stats: TextStatistics) -> float:
0104         if not text:
0105             return 0.0
0106         score = 40.0
0107         if stats.sentences >= 3:
0108             score += 25
0109         if stats.paragraphs >= 2:
0110             score += 20
0111         if text[-1] in ". ! ? !?...\"'\"' ] ) ":
0112             score += 10
0113         if re.search(r"[. , ; : ]", text):
0114             score += 5
0115         return min(100.0, score)
0116
0117     @staticmethod
0118     def _length_score(level: ReadingLevel, length: int) -> float:
0119         targets = {
0120             ReadingLevel.PRIMARY: (120, 500),
0121             ReadingLevel.JUNIOR: (250, 1000),
0122             ReadingLevel.SENIOR: (400, 1800),
0123         }
0124         minimum, maximum = targets[level]
0125         if minimum <= length <= maximum:
0126             return 100.0
0127         if length < minimum:
0128             return max(0.0, length / minimum * 100.0)
0129         overflow = (length - maximum) / maximum
0130         return max(40.0, 100.0 - overflow * 70.0)
0131
0132     def _grade(self, score: float) -> str:
0133         if score >= self.config.excellent_score:
0134             return "优秀"
0135         if score >= self.config.good_score:
0136             return "良好"
0137         if score >= self.config.pass_score:
0138             return "合格"
0139         return "待改进"
0140
0141     def _advice(
0142         self,
0143         text: str,
0144         target: ReadingLevel,
0145         stats: TextStatistics,
0146         metrics: QualityMetrics,
0147     ) -> tuple[list[str], list[str]]:
0148         warnings: list[str] = []
0149         suggestions: list[str] = []

```

```

0150         if not text:
0151             return ["文本为空"], ["请输入需要评估的完整中文文本"]
0152         if metrics.repetition_rate > self.config.repetition_warning:
0153             warnings.append("连续片段重复比例偏高")
0154             suggestions.append("删除机械重复句, 使用指代或改写衔接相近内容")
0155         if metrics.english_ratio > self.config.english_warning:
0156             warnings.append("英文字符比例偏高")
0157             suggestions.append("确认英文术语是否必要, 并在首次出现时提供中文解释")
0158         sentence_targets = {
0159             ReadingLevel.PRIMARY: self.config.primary_sentence_target,
0160             ReadingLevel.JUNIOR: self.config.junior_sentence_target,
0161             ReadingLevel.SENIOR: self.config.senior_sentence_target,
0162         }
0163         if stats.average_sentence_length > sentence_targets[target] * 1.35:
0164             warnings.append("平均句长高于目标等级建议值")
0165             suggestions.append("拆分长句, 减少连续修饰成分和多层转折")
0166         if stats.paragraphs < 2 and stats.chinese_characters >= 100:
0167             warnings.append("长文本缺少段落划分")
0168             suggestions.append("按引入、说明和总结划分自然段")
0169         if metrics.length_score < 70:
0170             warnings.append("文本长度与目标等级不匹配")
0171             suggestions.append("围绕主题补充事实、例子或总结, 避免无关扩写")
0172         if metrics.level_score < 80:
0173             warnings.append("预测难度与目标阅读等级存在差异")
0174             direction = "降低" if stats.average_sentence_length > sentence_targets[target] else "提高"
0175             suggestions.append(f"通过词语解释、句式调整和信息密度控制来{direction}阅读难度")
0176         return warnings, list(dict.fromkeys(suggestions))
0177
===== 文件开始 src/llmctg/generators/__init__.py =====
0001 from .base import TextGenerator
0002 from .factory import create_generator
0003 from .template import TemplateGenerator
0004
0005 __all__ = ["TextGenerator", "TemplateGenerator", "create_generator"]
0006
===== 文件开始 src/llmctg/generators/base.py =====
0001 from __future__ import annotations
0002
0003 from abc import ABC, abstractmethod
0004
0005 from ..models import GenerationRequest
0006
0007
0008 class TextGenerator(ABC):
0009     name = "base"
0010
0011     @abstractmethod
0012     def generate(self, request: GenerationRequest) -> list[tuple[str, str]]:
0013         """返回标题、正文组成的列表。"""
0014

```

源程序第 22 页 src/llmctg/generators/base.py

```
0015     def close(self) -> None:
0016         """释放可选的模型资源。"""
0017
0018 ===== 文件开始 src/llmctg/generators/factory.py =====
0001 from __future__ import annotations
0002
0003 from ..config import GenerationConfig
0004 from .base import TextGenerator
0005 from .template import TemplateGenerator
0006
0007
0008 def create_generator(config: GenerationConfig) -> TextGenerator:
0009     backend = config.backend.strip().lower()
0010     if backend in {"template", "demo", "offline"}:
0011         return TemplateGenerator()
0012     if backend in {"transformers", "huggingface", "local-model"}:
0013         from .transformers_backend import TransformersGenerator
0014         return TransformersGenerator(config)
0015     raise ValueError(f"不支持的生成后端: {config.backend}")
0016
0017 ===== 文件开始 src/llmctg/generators/template.py =====
0001 from __future__ import annotations
0002
0003 import random
0004
0005 from ..models import GenerationRequest, ReadingLevel
0006 from ..text_utils import ensure_terminal_punctuation
0007 from .base import TextGenerator
0008
0009
0010 class TemplateGenerator(TextGenerator):
0011     """用于离线演示和安装验收的确定性生成器。"""
0012
0013     name = "template"
0014
0015     OPENINGS = {
0016         ReadingLevel.PRIMARY: [
0017             "今天，我们一起认识{topic}。",
0018             "你注意过{topic}吗？它就在我们的生活里。",
0019             "关于{topic}，有许多有趣的小秘密。",
0020         ],
0021         ReadingLevel.JUNIOR: [
0022             "当我们讨论{topic}时，首先要理解它与日常生活的联系。",
0023             "{topic}看似熟悉，其中却包含值得追问的规律。",
0024             "从一个常见现象出发，我们可以逐步认识{topic}。",
0025         ],
0026         ReadingLevel.SENIOR: [
0027             "理解{topic}，需要同时观察事实、机制及其产生的社会影响。",
0028             "{topic}并非孤立现象，它由多种条件共同塑造。",
0029             "围绕{topic}的讨论，往往涉及科学判断与价值选择的结合。",
```

源程序第 23 页 src/llmctg/generators/template.py

```
0030     ],
0031 }
0032 OBSERVATIONS = {
0033     ReadingLevel.PRIMARY: [
0034         "我们可以先看一看，再想一想它为什么会这样。",
0035         "仔细观察，会发现变化不是一下子发生的。",
0036         "把看到的事情记下来，就能发现新的线索。",
0037     ],
0038     ReadingLevel.JUNIOR: [
0039         "观察现象只是第一步，还要比较条件、记录变化并寻找原因。",
0040         "同一现象在不同环境中可能出现差异，因此结论需要证据支持。",
0041         "提出问题、收集资料和验证猜想，可以帮助我们避免凭感觉判断。",
0042     ],
0043     ReadingLevel.SENIOR: [
0044         "分析这一主题时，应区分相关关系与因果关系，并说明证据的适用范围。",
0045         "可靠的结论来自可重复的观察、清晰的概念和对反例的认真检验。",
0046         "不同尺度上的机制可能彼此影响，单一解释通常不足以覆盖全部情形。",
0047     ],
0048 }
0049 ACTIONS = {
0050     ReadingLevel.PRIMARY: [
0051         "我们可以从身边的小事做起，把问题写进观察本。",
0052         "遇到不懂的地方，可以查资料，也可以请教老师和同学。",
0053         "只要愿意动手试一试，知识就会变得更清楚。",
0054     ],
0055     ReadingLevel.JUNIOR: [
0056         "实践时可以设置一个明确目标，并用相同标准记录每次结果。",
0057         "阅读多种资料、核对来源，再用自己的语言概括，是有效的学习方法。",
0058         "如果结果与预期不同，不妨检查步骤并修正原来的假设。",
0059     ],
0060     ReadingLevel.SENIOR: [
0061         "进一步研究可以建立指标体系，比较不同方案的收益、成本与不确定性。",
0062         "面对相互冲突的材料，应检查样本、方法和论证链条，而非只看结论。",
0063         "将理论解释转化为可检验的问题，有助于形成审慎而开放的判断。",
0064     ],
0065 }
0066 CLOSINGS = {
0067     ReadingLevel.PRIMARY: [
0068         "让我们带着好奇心，继续发现{topic}的故事吧！",
0069         "下一次见到它时，你也许会有新的发现。",
0070     ],
0071     ReadingLevel.JUNIOR: [
0072         "由此可见，认识{topic}既需要知识，也需要耐心和实践。",
0073         "持续观察和主动求证，会让我们对{topic}形成更完整的认识。",
0074     ],
0075     ReadingLevel.SENIOR: [
0076         "因此，对{topic}的理解应随着证据更新，并在具体情境中接受检验。",
0077         "只有把事实、逻辑与责任结合起来，我们才能更稳妥地回应相关问题。",
0078     ],
0079 }
```

```

0080
0081 def generate(self, request: GenerationRequest) -> list[tuple[str, str]]:
0082     request.validate()
0083     randomizer = random.Random(request.seed)
0084     results: list[tuple[str, str]] = []
0085     for index in range(request.count):
0086         title = self._title(request, index)
0087         paragraphs = self._paragraphs(request, randomizer)
0088         results.append((title, ensure_terminal_punctuation("\n".join(paragraphs))))
0089     return results
0090
0091 def _title(self, request: GenerationRequest, index: int) -> str:
0092     suffix = "" if request.count == 1 else f" ({index + 1}) "
0093     names = {
0094         ReadingLevel.PRIMARY: f"认识{request.topic}",
0095         ReadingLevel.JUNIOR: f"探索{request.topic}",
0096         ReadingLevel.SENIOR: f"理解{request.topic}的多重视角",
0097     }
0098     return names[request.level] + suffix
0099
0100 def _paragraphs(self, request: GenerationRequest, randomizer: random.Random) -> list[str]:
0101     context = {"topic": request.topic}
0102     opening = randomizer.choice(self.OPENINGS[request.level]).format(**context)
0103     observation = randomizer.choice(self.OBSERVATIONS[request.level]).format(**context)
0104     action = randomizer.choice(self.ACTIONS[request.level]).format(**context)
0105     closing = randomizer.choice(self.CLOSINGS[request.level]).format(**context)
0106     keyword_sentence = self._keyword_sentence(request)
0107     instruction_sentence = self._instruction_sentence(request)
0108     if request.level == ReadingLevel.PRIMARY:
0109         return [opening + keyword_sentence, observation, action, closing]
0110     if request.level == ReadingLevel.JUNIOR:
0111         return [opening + keyword_sentence, observation + instruction_sentence, action, closing]
0112     extension = (
0113         f"以{request.topic}为例，个体经验能够提出问题，但仍需通过系统资料来检验。"
0114         "当条件改变时，原有解释也可能需要调整，这正是理性探究的价值所在。"
0115     )
0116     return [opening + keyword_sentence, observation + instruction_sentence, extension, action, closing]
0117
0118 @staticmethod
0119 def _keyword_sentence(request: GenerationRequest) -> str:
0120     if not request.keywords:
0121         return ""
0122     joined = "、".join(request.keywords[:6])
0123     return f"阅读时可以留意这些关键词：{joined}。"
0124
0125 @staticmethod
0126 def _instruction_sentence(request: GenerationRequest) -> str:
0127     if not request.instruction.strip():
0128         return ""
0129     return f"本次阅读任务是：{request.instruction.strip()}。"

```

```

0130
===== 文件开始 src/llmctg/generators/transformers_backend.py =====
0001 from __future__ import annotations
0002
0003 from pathlib import Path
0004 from typing import Any
0005
0006 from ..config import GenerationConfig
0007 from ..models import GenerationRequest, ReadingLevel
0008 from ..text_utils import ensure_terminal_punctuation
0009 from .base import TextGenerator
0010
0011
0012 class TransformersGenerator(TextGenerator):
0013     """惰性加载本地 Hugging Face 模型的可选生成后端。"""
0014
0015     name = "transformers"
0016
0017     def __init__(self, config: GenerationConfig):
0018         if not config.model_path:
0019             raise ValueError("使用 transformers 后端时必须指定 model_path")
0020         self.config = config
0021         self._torch: Any = None
0022         self._tokenizer: Any = None
0023         self._model: Any = None
0024
0025     def _load(self) -> None:
0026         if self._model is not None:
0027             return
0028         try:
0029             import torch
0030             from transformers import AutoModelForCausalLM, AutoTokenizer
0031         except ImportError as error:
0032             raise RuntimeError("缺少模型依赖, 请执行 pip install -e .[llm]") from error
0033         self._torch = torch
0034         self._tokenizer = AutoTokenizer.from_pretrained(
0035             self.config.model_path, trust_remote_code=False, use_fast=True
0036         )
0037         device = self.config.device
0038         if device == "auto":
0039             device = "cuda" if torch.cuda.is_available() else "cpu"
0040         dtype = torch.float16 if str(device).startswith("cuda") else torch.float32
0041         model = AutoModelForCausalLM.from_pretrained(
0042             self.config.model_path, trust_remote_code=False, dtype=dtype
0043         )
0044         if self.config.lora_path:
0045             adapter_path = Path(self.config.lora_path)
0046             if not adapter_path.is_dir():
0047                 raise FileNotFoundError(f"LoRA 适配器目录不存在: {adapter_path}")
0048             if not (adapter_path / "adapter_config.json").is_file():

```

源程序第 26 页 src/llmctg/generators/transformers_backend.py

```
0049         raise FileNotFoundError(f"LoRA 适配器配置不存在: {adapter_path / 'adapter_config.json'}")
0050     try:
0051         from peft import PeftModel
0052     except ImportError as error:
0053         raise RuntimeError("加载 LoRA 适配器需要 peft, 请执行 pip install -e .[llm]") from error
0054     model = PeftModel.from_pretrained(
0055         model, str(adapter_path), is_trainable=False
0056     )
0057     self._model = model.to(device)
0058     self._model.eval()
0059     if self._tokenizer.pad_token_id is None:
0060         self._tokenizer.pad_token = self._tokenizer.eos_token
0061
0062     def generate(self, request: GenerationRequest) -> list[tuple[str, str]]:
0063         request.validate()
0064         self._load()
0065         return [self._generate_one(request, index) for index in range(request.count)]
0066
0067     def _generate_one(self, request: GenerationRequest, index: int) -> tuple[str, str]:
0068         prompt = self._build_prompt(request)
0069         device = next(self._model.parameters()).device
0070         encoded = self._tokenizer(prompt, return_tensors="pt", truncation=True, max_length=2048)
0071         encoded = {key: value.to(device) for key, value in encoded.items()}
0072         maximums = {
0073             ReadingLevel.PRIMARY: self.config.max_new_tokens_primary,
0074             ReadingLevel.JUNIOR: self.config.max_new_tokens_junior,
0075             ReadingLevel.SENIOR: self.config.max_new_tokens_senior,
0076         }
0077         if request.seed is not None:
0078             seed = request.seed + index
0079             self._torch.manual_seed(seed)
0080             if self._torch.cuda.is_available():
0081                 self._torch.cuda.manual_seed_all(seed)
0082         with self._torch.inference_mode():
0083             output = self._model.generate(
0084                 **encoded,
0085                 do_sample=True,
0086                 max_new_tokens=maximums[request.level],
0087                 temperature=self.config.temperature,
0088                 top_p=self.config.top_p,
0089                 top_k=self.config.top_k,
0090                 repetition_penalty=self.config.repetition_penalty,
0091                 pad_token_id=self._tokenizer.pad_token_id,
0092                 eos_token_id=self._tokenizer.eos_token_id,
0093             )
0094         new_tokens = output[0, encoded["input_ids"].shape[-1]:]
0095         content = self._tokenizer.decode(new_tokens, skip_special_tokens=True).strip()
0096         title = f"{request.topic}分级阅读材料"
0097         return title, ensure_terminal_punctuation(content)
0098
```

源程序第 27 页 src/llmctg/generators/transformers_backend.py

```
0099     @staticmethod
0100     def _build_prompt(request: GenerationRequest) -> str:
0101         keywords = "、".join(request.keywords) if request.keywords else "无指定关键词"
0102         extra = request.instruction.strip() or "内容准确、结构完整、表达自然"
0103         return (
0104             f"请围绕“{request.topic}”创作一篇适合{request.level.chinese_name}学生阅读的中文文章。\\n"
0105             f"关键词: {keywords}\\n 要求: {extra}\\n 只输出文章正文, 不要解释创作过程。\\n 正文: "
0106         )
0107
0108     def close(self) -> None:
0109         self._model = None
0110         self._tokenizer = None
0111         if self._torch is not None and self._torch.cuda.is_available():
0112             self._torch.cuda.empty_cache()
0113
0114
0115 ===== 文件开始 src/llmctg/io_utils.py =====
0116
0117 from __future__ import annotations
0118
0119 import csv
0120 import json
0121 from pathlib import Path
0122 from typing import Any, Iterable, Iterator
0123
0124 class DataFormatError(ValueError):
0125     """输入记录无法解析时抛出。"""
0126
0127
0128 def read_jsonl(path: str | Path) -> Iterator[dict[str, Any]]:
0129     source = Path(path)
0130     if not source.exists():
0131         raise FileNotFoundError(f"输入文件不存在: {source}")
0132     with source.open("r", encoding="utf-8-sig") as stream:
0133         for line_number, line in enumerate(stream, 1):
0134             stripped = line.strip()
0135             if not stripped:
0136                 continue
0137             try:
0138                 value = json.loads(stripped)
0139             except json.JSONDecodeError as error:
0140                 raise DataFormatError(f"{source} 第 {line_number} 行不是有效 JSON: {error.msg}") from error
0141             if not isinstance(value, dict):
0142                 raise DataFormatError(f"{source} 第 {line_number} 行必须是 JSON 对象")
0143             value["_line_number"] = line_number
0144             yield value
0145
0146
0147 def append_jsonl(path: str | Path, value: dict[str, Any]) -> Path:
0148     target = Path(path)
```

源程序第 28 页 src/llmctg/io_utils.py

```
0034     target.parent.mkdir(parents=True, exist_ok=True)
0035     with target.open("a", encoding="utf-8", newline="\n") as stream:
0036         stream.write(json.dumps(value, ensure_ascii=False, separators=(",", ":")) + "\n")
0037     return target
0038
0039
0040 def write_json(path: str | Path, value: Any) -> Path:
0041     target = Path(path)
0042     target.parent.mkdir(parents=True, exist_ok=True)
0043     with target.open("w", encoding="utf-8", newline="\n") as stream:
0044         json.dump(value, stream, ensure_ascii=False, indent=2)
0045         stream.write("\n")
0046     return target
0047
0048
0049 def write_csv(path: str | Path, rows: Iterable[dict[str, Any]], fieldnames: list[str]) -> Path:
0050     target = Path(path)
0051     target.parent.mkdir(parents=True, exist_ok=True)
0052     with target.open("w", encoding="utf-8-sig", newline="") as stream:
0053         writer = csv.DictWriter(stream, fieldnames=fieldnames, extrasaction="ignore")
0054         writer.writeheader()
0055         writer.writerows(rows)
0056     return target
0057
0058
0059 def existing_ids(path: str | Path, key: str = "request_id") -> set[str]:
0060     target = Path(path)
0061     if not target.exists():
0062         return set()
0063     found: set[str] = set()
0064     try:
0065         for record in read_jsonl(target):
0066             value = record.get(key)
0067             if value:
0068                 found.add(str(value))
0069     except DataFormatError:
0070         return found
0071     return found
0072
0073
0074 ===== 文件开始 src/llmctg/logging_utils.py =====
0001 from __future__ import annotations
0002
0003 import logging
0004 from pathlib import Path
0005
0006
0007 def configure_logging(level: str = "INFO", log_file: str | Path | None = None) -> None:
0008     handlers: list[logging.Handler] = [logging.StreamHandler()]
0009     if log_file:
0010         target = Path(log_file)
```

源程序第 29 页 src/llmctg/logging_utils.py

```
0011         target.parent.mkdir(parents=True, exist_ok=True)
0012         handlers.append(logging.FileHandler(target, encoding="utf-8"))
0013     logging.basicConfig(
0014         level=getattr(logging, level.upper(), logging.INFO),
0015         format="%(asctime)s %(levelname)s %(name)s %(message)s",
0016         handlers=handlers,
0017         force=True,
0018     )
0019
0020 ===== 文件开始 src/llmctg/models.py =====
0021 from __future__ import annotations
0022
0023 from dataclasses import asdict, dataclass, field
0024 from enum import Enum
0025 from typing import Any
0026
0027
0028 class ReadingLevel(str, Enum):
0029     PRIMARY = "primary"
0030     JUNIOR = "junior"
0031     SENIOR = "senior"
0032
0033     @property
0034     def chinese_name(self) -> str:
0035         return {self.PRIMARY: "小学", self.JUNIOR: "初中", self.SENIOR: "高中"}[self]
0036
0037     @property
0038     def numeric_label(self) -> int:
0039         return {self.PRIMARY: 0, self.JUNIOR: 1, self.SENIOR: 2}[self]
0040
0041     @classmethod
0042     def parse(cls, value: str | int | "ReadingLevel") -> "ReadingLevel":
0043         if isinstance(value, cls):
0044             return value
0045         aliases = {
0046             "0": cls.PRIMARY, "小学": cls.PRIMARY, "小学生": cls.PRIMARY,
0047             "primary": cls.PRIMARY,
0048             "1": cls.JUNIOR, "初中": cls.JUNIOR, "初中生": cls.JUNIOR,
0049             "junior": cls.JUNIOR,
0050             "2": cls.SENIOR, "高中": cls.SENIOR, "高中生": cls.SENIOR,
0051             "senior": cls.SENIOR,
0052         }
0053         key = str(value).strip().lower()
0054         if key not in aliases:
0055             raise ValueError(f"不支持的阅读等级: {value}")
0056         return aliases[key]
0057
0058
0059 @dataclass(slots=True)
0060 class GenerationRequest:
```

```

0041     topic: str
0042     level: ReadingLevel = ReadingLevel.JUNIOR
0043     instruction: str = ""
0044     keywords: list[str] = field(default_factory=list)
0045     count: int = 1
0046     seed: int | None = None
0047
0048     def validate(self) -> None:
0049         self.topic = self.topic.strip()
0050         if not self.topic:
0051             raise ValueError("主题不能为空")
0052         if not 1 <= self.count <= 20:
0053             raise ValueError("单次生成数量必须在 1 到 20 之间")
0054         self.level = ReadingLevel.parse(self.level)
0055         self.keywords = [str(item).strip() for item in self.keywords if str(item).strip()]
0056
0057
0058 @dataclass(slots=True)
0059 class TextStatistics:
0060     characters: int
0061     chinese_characters: int
0062     sentences: int
0063     paragraphs: int
0064     average_sentence_length: float
0065     max_sentence_length: int
0066     unique_character_ratio: float
0067     punctuation_ratio: float
0068
0069
0070 @dataclass(slots=True)
0071 class QualityMetrics:
0072     repetition_rate: float
0073     english_ratio: float
0074     uncommon_character_ratio: float
0075     structure_score: float
0076     length_score: float
0077     level_score: float
0078     overall_score: float
0079
0080
0081 @dataclass(slots=True)
0082 class EvaluationResult:
0083     target_level: ReadingLevel
0084     predicted_level: ReadingLevel
0085     quality_grade: str
0086     passed: bool
0087     statistics: TextStatistics
0088     metrics: QualityMetrics
0089     warnings: list[str] = field(default_factory=list)
0090     suggestions: list[str] = field(default_factory=list)

```

```

0091
0092     def to_dict(self) -> dict[str, Any]:
0093         data = asdict(self)
0094         data["target_level"] = self.target_level.value
0095         data["predicted_level"] = self.predicted_level.value
0096         return data
0097
0098
0099 @dataclass(slots=True)
0100 class GeneratedArticle:
0101     request_id: str
0102     topic: str
0103     level: ReadingLevel
0104     title: str
0105     content: str
0106     backend: str
0107     created_at: str
0108     evaluation: EvaluationResult | None = None
0109
0110     def to_dict(self) -> dict[str, Any]:
0111         data = asdict(self)
0112         data["level"] = self.level.value
0113         if self.evaluation:
0114             data["evaluation"] = self.evaluation.to_dict()
0115         return data
0116
0117 ===== 文件开始 src/llmctg/pipeline.py =====
0001 from __future__ import annotations
0002
0003 import logging
0004 from datetime import datetime, timezone
0005 from pathlib import Path
0006 from typing import Any, Iterable
0007
0008 from .config import AppConfig
0009 from .evaluator import ReadabilityEvaluator
0010 from .generators import TextGenerator, create_generator
0011 from .io_utils import append_jsonl, existing_ids, read_jsonl, write_json
0012 from .models import GeneratedArticle, GenerationRequest, ReadingLevel
0013 from .reporting import BatchSummary, build_summary, write_html_report, write_summary_csv
0014 from .text_utils import stable_id
0015
0016 LOGGER = logging.getLogger(__name__)
0017
0018
0019 class LLMCTGApplication:
0020     def __init__(self, config: AppConfig | None = None, generator: TextGenerator | None = None):
0021         self.config = config or AppConfig()
0022         self.generator = generator or create_generator(self.config.generation)
0023         self.evaluator = ReadabilityEvaluator(self.config.evaluation)

```

```

0024
0025 def generate(self, request: GenerationRequest) -> list[GeneratedArticle]:
0026     request.validate()
0027     raw_results = self.generator.generate(request)
0028     created_at = datetime.now(timezone.utc).astimezone().isoformat(timespec="seconds")
0029     articles: list[GeneratedArticle] = []
0030     for index, (title, content) in enumerate(raw_results):
0031         identifier = stable_id(request.topic, request.level.value, request.seed, index, content)
0032         evaluation = self.evaluator.evaluate(content, request.level)
0033         articles.append(
0034             GeneratedArticle(
0035                 request_id=identifier,
0036                 topic=request.topic,
0037                 level=request.level,
0038                 title=title,
0039                 content=content,
0040                 backend=self.generator.name,
0041                 created_at=created_at,
0042                 evaluation=evaluation,
0043             )
0044         )
0045     return articles
0046
0047 def evaluate(self, text: str, level: ReadingLevel | str | int):
0048     return self.evaluator.evaluate(text, level)
0049
0050 def batch(self, input_path: str | Path, output_dir: str | Path, resume: bool = True) -> BatchSummary:
0051     output = Path(output_dir)
0052     output.mkdir(parents=True, exist_ok=True)
0053     records_path = output / "articles.jsonl"
0054     errors_path = output / "errors.jsonl"
0055     done = existing_ids(records_path) if resume else set()
0056     articles: list[GeneratedArticle] = []
0057     skipped = 0
0058     failed = 0
0059     for record in read_jsonl(input_path):
0060         source_id = stable_id(record.get("topic", ""), record.get("level", "junior"), record.get("_line_number"))
0061         if source_id in done:
0062             skipped += 1
0063             continue
0064         try:
0065             request = self._request_from_record(record)
0066             generated = self.generate(request)
0067             for article in generated:
0068                 payload = article.to_dict()
0069                 payload["source_id"] = source_id
0070                 append_jsonl(records_path, payload)
0071                 articles.append(article)
0072             done.add(source_id)
0073         except Exception as error: # 单条失败不终止批处理

```

源程序第 33 页 src/llmctg/pipeline.py

```
0074         failed += 1
0075         LOGGER.exception("第 %s 行处理失败", record.get("_line_number"))
0076         append_jsonl(errors_path, {
0077             "source_id": source_id,
0078             "line_number": record.get("_line_number"),
0079             "error_type": type(error).__name__,
0080             "message": str(error),
0081             "record": {key: value for key, value in record.items() if not key.startswith("_")},
0082         })
0083         summary = build_summary(articles, skipped=skipped, failed=failed)
0084         write_json(output / "summary.json", summary.to_dict())
0085         write_summary_csv(output / "summary.csv", articles)
0086         write_html_report(output / "report.html", articles, summary)
0087         return summary
0088
0089     @staticmethod
0090     def _request_from_record(record: dict[str, Any]) -> GenerationRequest:
0091         keywords = record.get("keywords", [])
0092         if isinstance(keywords, str):
0093             keywords = [item.strip() for item in keywords.replace(" ", "").split(",") if item.strip()]
0094         return GenerationRequest(
0095             topic=str(record.get("topic", "")),
0096             level=ReadingLevel.parse(record.get("level", "junior")),
0097             instruction=str(record.get("instruction", "")),
0098             keywords=list(keywords),
0099             count=int(record.get("count", 1)),
0100             seed=int(record.get("seed")) if record.get("seed") is not None else None,
0101         )
0102
0103     def close(self) -> None:
0104         self.generator.close()
0105
0106     def __enter__(self) -> "LLMCTGApplication":
0107         return self
0108
0109     def __exit__(self, exc_type: Any, exc: Any, traceback: Any) -> None:
0110         self.close()
0111
0112 ===== 文件开始 src/llmctg/reporting.py =====
0001 from __future__ import annotations
0002
0003 import html
0004 from dataclasses import asdict, dataclass
0005 from pathlib import Path
0006 from statistics import mean
0007
0008 from .io_utils import write_csv
0009 from .models import GeneratedArticle, ReadingLevel
0010 from .text_utils import write_text
0011
```

```

0012
0013 @dataclass(slots=True)
0014 class BatchSummary:
0015     generated: int
0016     passed: int
0017     failed_quality: int
0018     skipped: int
0019     processing_errors: int
0020     average_score: float
0021     primary_count: int
0022     junior_count: int
0023     senior_count: int
0024
0025     def to_dict(self) -> dict[str, int | float]:
0026         return asdict(self)
0027
0028
0029 def build_summary(
0030     articles: list[GeneratedArticle], skipped: int = 0, failed: int = 0
0031 ) -> BatchSummary:
0032     scores = [article.evaluation.metrics.overall_score for article in articles if article.evaluation]
0033     passed = sum(1 for article in articles if article.evaluation and article.evaluation.passed)
0034     return BatchSummary(
0035         generated=len(articles),
0036         passed=passed,
0037         failed_quality=len(articles) - passed,
0038         skipped=skipped,
0039         processing_errors=failed,
0040         average_score=round(mean(scores), 2) if scores else 0.0,
0041         primary_count=sum(article.level == ReadingLevel.PRIMARY for article in articles),
0042         junior_count=sum(article.level == ReadingLevel.JUNIOR for article in articles),
0043         senior_count=sum(article.level == ReadingLevel.SENIOR for article in articles),
0044     )
0045
0046
0047 def write_summary_csv(path: str | Path, articles: list[GeneratedArticle]) -> Path:
0048     rows = []
0049     for article in articles:
0050         evaluation = article.evaluation
0051         rows.append({
0052             "request_id": article.request_id,
0053             "topic": article.topic,
0054             "level": article.level.value,
0055             "title": article.title,
0056             "backend": article.backend,
0057             "quality_grade": evaluation.quality_grade if evaluation else "",
0058             "passed": evaluation.passed if evaluation else False,
0059             "overall_score": evaluation.metrics.overall_score if evaluation else "",
0060             "characters": evaluation.statistics.chinese_characters if evaluation else "",
0061             "predicted_level": evaluation.predicted_level.value if evaluation else "",

```

```

0062     })
0063     return write_csv(path, rows, [
0064         "request_id", "topic", "level", "title", "backend", "quality_grade",
0065         "passed", "overall_score", "characters", "predicted_level",
0066     ])
0067
0068
0069 def write_html_report(
0070     path: str | Path, articles: list[GeneratedArticle], summary: BatchSummary
0071 ) -> Path:
0072     cards = "\n".join(_article_card(article) for article in articles)
0073     document = f"""<!doctype html>
0074 <html lang="zh-CN">
0075 <head>
0076 <meta charset="utf-8">
0077 <meta name="viewport" content="width=device-width,initial-scale=1">
0078 <title>中文分级阅读内容质量报告</title>
0079 <style>
0080 body{{font-family:"Microsoft YaHei",sans-serif;margin:0;background:#f4f6f8;color:#1f2937}}
0081 main{{max-width:1080px;margin:0 auto;padding:32px 20px}}
0082 h1{{font-size:28px;margin:0 0 8px}} .sub{{color:#64748b;margin-bottom:24px}}
0083 .summary{{display:grid;grid-template-columns:repeat(auto-fit,minmax(150px,1fr));gap:12px;margin:20px 0 28px}}
0084 .metric,.card{{background:#fff;border:1px solid #e2e8f0;border-radius:10px;padding:16px}}
0085 .metric strong{{display:block;font-size:25px;color:#0f4c81}} .card{{margin:14px 0}}
0086 .meta{{color:#64748b;font-size:14px}} .content{{white-space:pre-wrap;line-height:1.85}}
0087 .pass{{color:#177245}} .fail{{color:#b42318}} ul{{line-height:1.7}}
0088 </style>
0089 </head>
0090 <body><main>
0091 <h1>中文分级阅读内容质量报告</h1><div class="sub">批量生成与自动评估结果</div>
0092 <section class="summary">
0093 <div class="metric"><strong>{summary.generated}</strong>生成数量</div>
0094 <div class="metric"><strong>{summary.passed}</strong>质量通过</div>
0095 <div class="metric"><strong>{summary.average_score}</strong>平均得分</div>
0096 <div class="metric"><strong>{summary.processing_errors}</strong>处理异常</div>
0097 </section>{cards or '<p>本次没有新增文章。</p>'}
0098 </main></body></html>"""
0099     return write_text(path, document)
0100
0101
0102 def _article_card(article: GeneratedArticle) -> str:
0103     evaluation = article.evaluation
0104     if evaluation is None:
0105         return ""
0106     status_class = "pass" if evaluation.passed else "fail"
0107     status = "通过" if evaluation.passed else "待改进"
0108     warnings = "".join(f"<li>{html.escape(item)}</li>" for item in evaluation.warnings)
0109     return f"""<article class="card">
0110 <h2>{html.escape(article.title)}</h2>
0111 <div class="meta">{html.escape(article.level.chinese_name)} · {html.escape(article.backend)} ·

```



```

0042
0043
0044 def character_ngrams(text: str, size: int = 4) -> list[str]:
0045     chars = chinese_characters(text)
0046     if size <= 0:
0047         raise ValueError("n-gram 长度必须为正整数")
0048     return ["".join(chars[index:index + size]) for index in range(max(0, len(chars) - size + 1))]
0049
0050
0051 def repetition_rate(text: str, size: int = 4) -> float:
0052     grams = character_ngrams(text, size=size)
0053     if not grams:
0054         return 0.0
0055     counts = Counter(grams)
0056     repeated = sum(count - 1 for count in counts.values() if count > 1)
0057     return repeated / len(grams)
0058
0059
0060 def stable_id(*values: object, length: int = 16) -> str:
0061     joined = "\x1f".join(str(value) for value in values)
0062     return hashlib.sha256(joined.encode("utf-8")).hexdigest()[:length]
0063
0064
0065 def safe_filename(value: str, fallback: str = "output") -> str:
0066     cleaned = re.sub(r"[<>:\\"/\\|]?*\x00-\x1f]", "_", normalize_text(value))
0067     cleaned = cleaned.strip("._")
0068     return (cleaned or fallback)[:80]
0069
0070
0071 def read_text(path: str | Path) -> str:
0072     with Path(path).open("r", encoding="utf-8-sig") as stream:
0073         return stream.read()
0074
0075
0076 def write_text(path: str | Path, content: str) -> Path:
0077     target = Path(path)
0078     target.parent.mkdir(parents=True, exist_ok=True)
0079     with target.open("w", encoding="utf-8", newline="\n") as stream:
0080         stream.write(content)
0081     return target
0082
0083
0084 ===== 文件开始 src/llmctg/training.py =====
0001 from __future__ import annotations
0002
0003 import inspect
0004 from dataclasses import dataclass
0005 from pathlib import Path
0006 from typing import Any
0007
0008

```

```

0009 @dataclass(slots=True)
0010 class FineTuneOptions:
0011     model_path: str
0012     dataset_path: str
0013     output_dir: str
0014     epochs: float = 3.0
0015     batch_size: int = 1
0016     gradient_accumulation_steps: int = 4
0017     learning_rate: float = 2e-4
0018     max_length: int = 1024
0019     lora_rank: int = 8
0020     lora_alpha: int = 32
0021     lora_dropout: float = 0.05
0022     device: str = "auto"
0023
0024     def validate(self) -> None:
0025         if not Path(self.model_path).exists():
0026             raise FileNotFoundError(f"基础模型目录不存在: {self.model_path}")
0027         if not Path(self.dataset_path).exists():
0028             raise FileNotFoundError(f"训练数据不存在: {self.dataset_path}")
0029         if self.epochs <= 0 or self.batch_size <= 0 or self.max_length < 64:
0030             raise ValueError("训练轮数、批量大小和最大长度必须为有效正数")
0031
0032
0033 class LoraTrainer:
0034     """把原始微调脚本封装为显式配置、可验证的训练服务。"""
0035
0036     def __init__(self, options: FineTuneOptions):
0037         options.validate()
0038         self.options = options
0039
0040     def train(self) -> dict[str, Any]:
0041         try:
0042             import torch
0043             from datasets import Dataset
0044             from peft import LoraConfig, TaskType, get_peft_model
0045             from transformers import (
0046                 AutoModelForCausalLM,
0047                 AutoTokenizer,
0048                 DataCollatorForLanguageModeling,
0049                 Trainer,
0050                 TrainingArguments,
0051             )
0052         except ImportError as error:
0053             raise RuntimeError("缺少训练依赖, 请安装 torch、transformers、datasets 和 peft") from error
0054
0055         device = self.options.device
0056         if device == "auto":
0057             device = "cuda" if torch.cuda.is_available() else "cpu"
0058         dtype = torch.float16 if device.startswith("cuda") else torch.float32

```

```

0059         tokenizer = AutoTokenizer.from_pretrained(
0060             self.options.model_path, trust_remote_code=False, use_fast=True
0061         )
0062         if tokenizer.pad_token_id is None:
0063             tokenizer.pad_token = tokenizer.eos_token
0064         base_model = AutoModelForCausalLM.from_pretrained(
0065             self.options.model_path, trust_remote_code=False, dtype=dtype
0066         )
0067         lora = LoraConfig(
0068             task_type=TaskType.CAUSAL_LM,
0069             inference_mode=False,
0070             r=self.options.lora_rank,
0071             lora_alpha=self.options.lora_alpha,
0072             lora_dropout=self.options.lora_dropout,
0073             target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
0074         )
0075         model = get_peft_model(base_model, lora)
0076         model.config.use_cache = False
0077         if device.startswith("cuda"):
0078             model.gradient_checkpointing_enable()
0079             model.enable_input_require_grads()
0080         model.to(device)
0081
0082         dataset = Dataset.from_json(self.options.dataset_path)
0083
0084         def tokenize(record: dict[str, Any]) -> dict[str, Any]:
0085             instruction = str(record.get("instruction", "")).strip()
0086             input_text = str(record.get("input", "")).strip()
0087             output_text = str(record.get("output", "")).strip()
0088             prompt = instruction + ("\n" + input_text if input_text else "")
0089             full_text = prompt + "\n" + output_text
0090             encoded = tokenizer(
0091                 full_text,
0092                 truncation=True,
0093                 max_length=self.options.max_length,
0094                 padding=False,
0095             )
0096             return encoded
0097
0098         tokenized = dataset.map(tokenize, remove_columns=dataset.column_names)
0099         argument_values = {
0100             "output_dir": self.options.output_dir,
0101             "overwrite_output_dir": True,
0102             "num_train_epochs": self.options.epochs,
0103             "per_device_train_batch_size": self.options.batch_size,
0104             "gradient_accumulation_steps": self.options.gradient_accumulation_steps,
0105             "learning_rate": self.options.learning_rate,
0106             "fp16": device.startswith("cuda"),
0107             "save_strategy": "epoch",
0108             "save_total_limit": 2,

```

源程序第 40 页 src/llmctg/training.py

```
0109         "logging_steps": 10,
0110         "report_to": "none",
0111         "dataloader_num_workers": 0,
0112         "remove_unused_columns": False,
0113     }
0114     supported_arguments = inspect.signature(TrainingArguments.__init__).parameters
0115     arguments = TrainingArguments(**{
0116         name: value
0117         for name, value in argument_values.items()
0118         if name in supported_arguments
0119     })
0120     trainer_values = {
0121         "model": model,
0122         "args": arguments,
0123         "train_dataset": tokenized,
0124         "data_collator": DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False),
0125     }
0126     supported_trainer_arguments = inspect.signature(Trainer.__init__).parameters
0127     if "processing_class" in supported_trainer_arguments:
0128         trainer_values["processing_class"] = tokenizer
0129     elif "tokenizer" in supported_trainer_arguments:
0130         trainer_values["tokenizer"] = tokenizer
0131     trainer = Trainer(**trainer_values)
0132     result = trainer.train()
0133     model.save_pretrained(self.options.output_dir)
0134     tokenizer.save_pretrained(self.options.output_dir)
0135     return {
0136         "output_dir": str(Path(self.options.output_dir).resolve()),
0137         "train_samples": len(tokenized),
0138         "train_loss": float(result.training_loss),
0139         "global_step": int(result.global_step),
0140         "device": device,
0141     }
0142
0143
0144 ===== 文件开始 tests/test_evaluator.py =====
0001 from llmctg.evaluator import ReadabilityEvaluator
0002 from llmctg.models import ReadingLevel
0003
0004
0005 def test_empty_text_fails():
0006     result = ReadabilityEvaluator().evaluate("", ReadingLevel.PRIMARY)
0007     assert not result.passed
0008     assert "文本为空" in result.warnings
0009
0010
0011 def test_evaluation_has_statistics():
0012     text = "春天来了。小草从泥土里探出头。我们走到公园，发现树枝上长出了新叶。\\n 大家把看到的变化写进观察本。"
0013     result = ReadabilityEvaluator().evaluate(text, ReadingLevel.PRIMARY)
0014     assert result.statistics.sentences == 4
```

源程序第 41 页 tests/test_evaluator.py

```
0015     assert result.statistics.paragraphs == 2
0016     assert 0 <= result.metrics.overall_score <= 100
0017
0018
0019 def test_repetition_warning():
0020     text = "春风吹来了，春风吹来了，春风吹来了，春风吹来了。\\n 我们听见春风吹来了。"
0021     result = ReadabilityEvaluator().evaluate(text, ReadingLevel.PRIMARY)
0022     assert result.metrics.repetition_rate > 0
0023     assert any("重复" in item for item in result.warnings)
0024
0025 ===== 文件开始 tests/test_lora_generation.py =====
0001 from pathlib import Path
0002
0003 import pytest
0004
0005 from llmctg.cli import build_parser
0006 from llmctg.config import AppConfig, GenerationConfig
0007 from llmctg.generators.transformers_backend import TransformersGenerator
0008
0009
0010 def test_generate_parser_accepts_lora_path():
0011     args = build_parser().parse_args([
0012         "generate",
0013         "--backend", "transformers",
0014         "--model-path", "/models/qwen",
0015         "--lora-path", "/adapters/chinese",
0016         "--topic", "城市河流",
0017     ])
0018     assert args.lora_path == "/adapters/chinese"
0019
0020
0021 def test_config_accepts_lora_path(tmp_path: Path):
0022     config_file = tmp_path / "config.json"
0023     config_file.write_text(
0024         '{"generation":{"backend":"transformers","model_path":"/model",'
0025         '"lora_path":"/adapter"}}',
0026         encoding="utf-8",
0027     )
0028     assert AppConfig.load(config_file).generation.lora_path == "/adapter"
0029
0030
0031 def test_missing_lora_directory_has_clear_error(monkeypatch, tmp_path: Path):
0032     generator = TransformersGenerator(GenerationConfig(
0033         backend="transformers",
0034         model_path=str(tmp_path / "model"),
0035         lora_path=str(tmp_path / "missing-adapter"),
0036         device="cpu",
0037     ))
0038
0039     class FakeTorch:
```

源程序第 42 页 tests/test_lora_generation.py

```
0040         float16 = "float16"
0041         float32 = "float32"
0042
0043         class cuda:
0044             @staticmethod
0045             def is_available():
0046                 return False
0047
0048         class FakeTokenizer:
0049             @staticmethod
0050             def from_pretrained(*args, **kwargs):
0051                 return object()
0052
0053         class FakeModelFactory:
0054             @staticmethod
0055             def from_pretrained(*args, **kwargs):
0056                 return object()
0057
0058         monkeypatch.setitem(__import__("sys").modules, "torch", FakeTorch)
0059         fake_transformers = type("FakeTransformers", (), {
0060             "AutoModelForCausalLM": FakeModelFactory,
0061             "AutoTokenizer": FakeTokenizer,
0062         })
0063         monkeypatch.setitem(__import__("sys").modules, "transformers", fake_transformers)
0064
0065
0066 ===== 文件开始 tests/test_models.py =====
0067 import pytest
0068
0069 from llmctg.models import GenerationRequest, ReadingLevel
0070
0071 def test_reading_level_aliases():
0072     assert ReadingLevel.parse("/小学") is ReadingLevel.PRIMARY
0073     assert ReadingLevel.parse("1") is ReadingLevel.JUNIOR
0074     assert ReadingLevel.parse("SENIOR") is ReadingLevel.SENIOR
0075
0076 def test_invalid_reading_level():
0077     with pytest.raises(ValueError):
0078         ReadingLevel.parse("大学")
0079
0080 def test_request_validation():
0081     request = GenerationRequest(topic=" 海洋  ", level="初中", keywords=[" 生物 ", ""])
0082     request.validate()
0083     assert request.topic == "海洋"
0084     assert request.level is ReadingLevel.JUNIOR
0085     assert request.keywords == ["生物"]
```

源程序第 43 页 tests/test_models.py

```
0024
0025 @pytest.mark.parametrize("count", [0, 21])
0026 def test_request_count_range(count):
0027     with pytest.raises(ValueError):
0028         GenerationRequest(topic="测试", count=count).validate()
0029
0030 ===== 文件开始 tests/test_pipeline.py =====
0001 import json
0002
0003 from llmctg.config import AppConfig
0004 from llmctg.models import GenerationRequest, ReadingLevel
0005 from llmctg.pipeline import LLMCTGApplication
0006
0007
0008 def test_generate_offline_article():
0009     with LLMCTGApplication(AppConfig()) as studio:
0010         articles = studio.generate(GenerationRequest(topic="月亮", level=ReadingLevel.PRIMARY, seed=7))
0011         assert len(articles) == 1
0012         assert articles[0].topic == "月亮"
0013         assert articles[0].evaluation is not None
0014
0015
0016 def test_batch_continues_after_bad_record(tmp_path):
0017     source = tmp_path / "tasks.jsonl"
0018     source.write_text(
0019         "\n".join([
0020             json.dumps({"topic": "河流", "level": "junior"}, ensure_ascii=False),
0021             json.dumps({"topic": "", "level": "junior"}, ensure_ascii=False),
0022         ]),
0023     encoding="utf-8",
0024     )
0025     with LLMCTGApplication(AppConfig()) as studio:
0026         summary = studio.batch(source, tmp_path / "output")
0027         assert summary.generated == 1
0028         assert summary.processing_errors == 1
0029         assert (tmp_path / "output" / "report.html").exists()
0030
0031 ===== 文件开始 tests/test_text_utils.py =====
0001 from llmctg.text_utils import (
0002     ensure_terminal_punctuation,
0003     normalize_text,
0004     repetition_rate,
0005     safe_filename,
0006     split_sentences,
0007     stable_id,
0008 )
0009
0010
0011 def test_normalize_text():
0012     assert normalize_text(" 春天 来了\r\n\r\n 小草发芽 ") == "春天 来了\n 小草发芽"
```

源程序第 44 页 tests/test_text_utils.py

```
0013
0014
0015 def test_sentence_helpers():
0016     text = ensure_terminal_punctuation("春天来了。小草发芽")
0017     assert text.endswith("。")
0018     assert len(split_sentences(text)) == 2
0019
0020
0021 def test_repetition_rate_detects_duplicate_ngram():
0022     assert repetition_rate("春风吹春风吹春风吹", size=3) > 0
0023     assert repetition_rate("春夏秋冬", size=4) == 0
0024
0025
0026 def test_stable_id_is_deterministic():
0027     assert stable_id("a", 1) == stable_id("a", 1)
0028     assert stable_id("a", 1) != stable_id("a", 2)
0029
0030
0031 def test_safe_filename():
0032     assert safe_filename('报告:"测试"/—') == "报告__测试__—"
0033
```